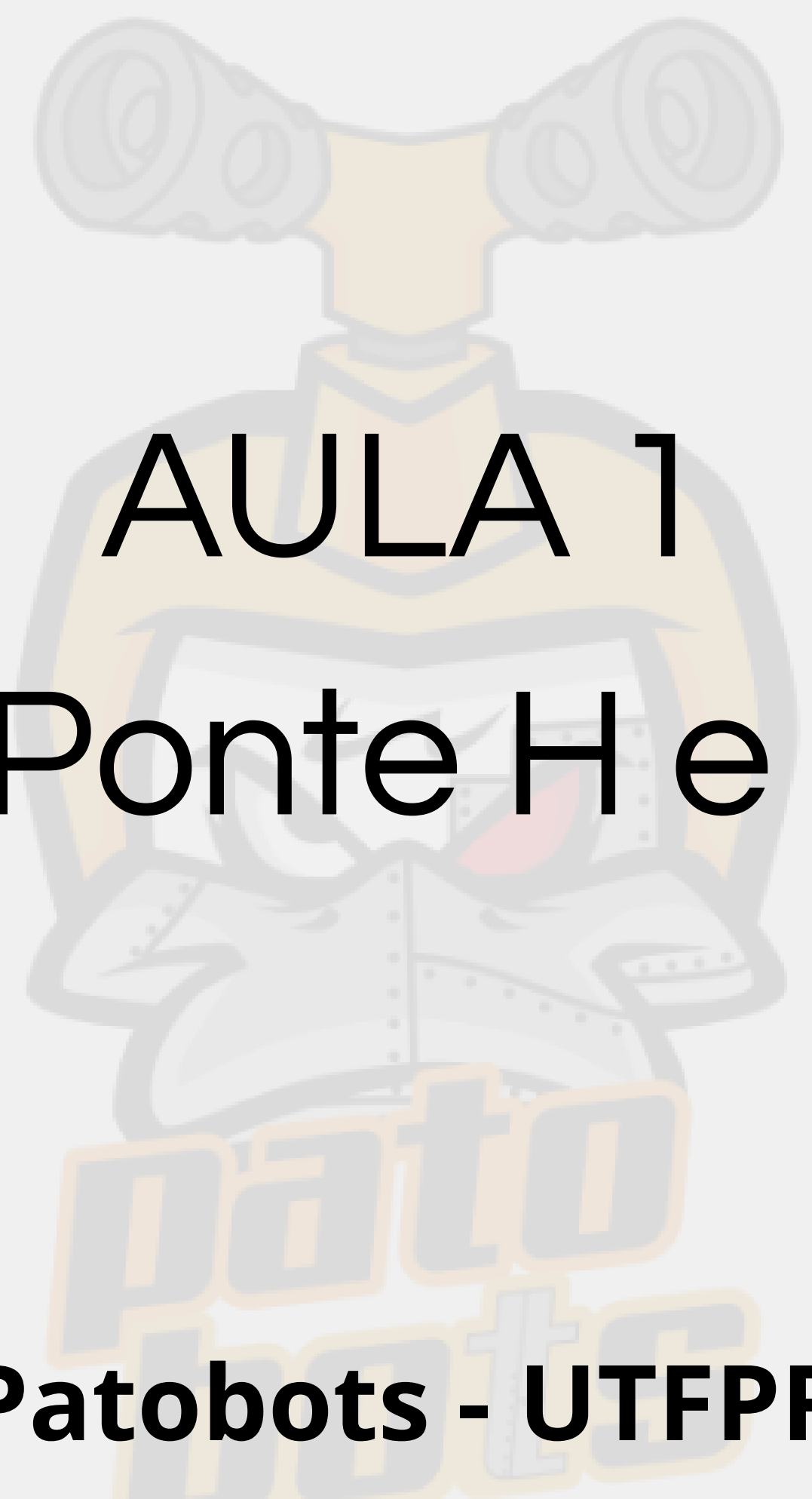




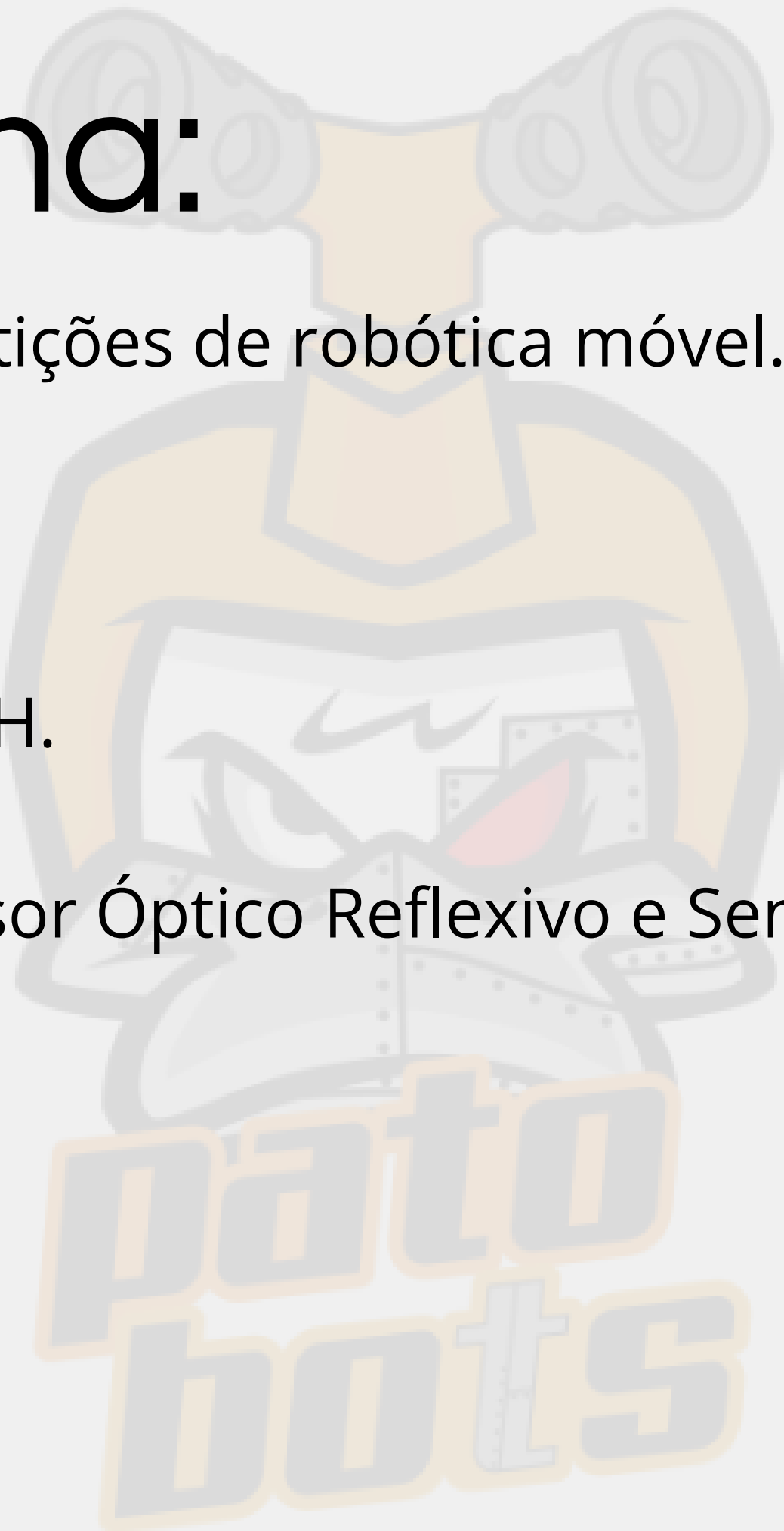
AULA 1

Motores, Ponte H e Sensores

Patobots - UTFPR

Cronograma:

- Modalidades de competições de robótica móvel.
- Motores.
- Acionamento do Motor.
- Ponte H.
- Acionamento da Ponte H.
- Step-Up.
- Tipos de Sensores: Sensor Óptico Reflexivo e Sensor QTR.
- Leitura dos Sensores.

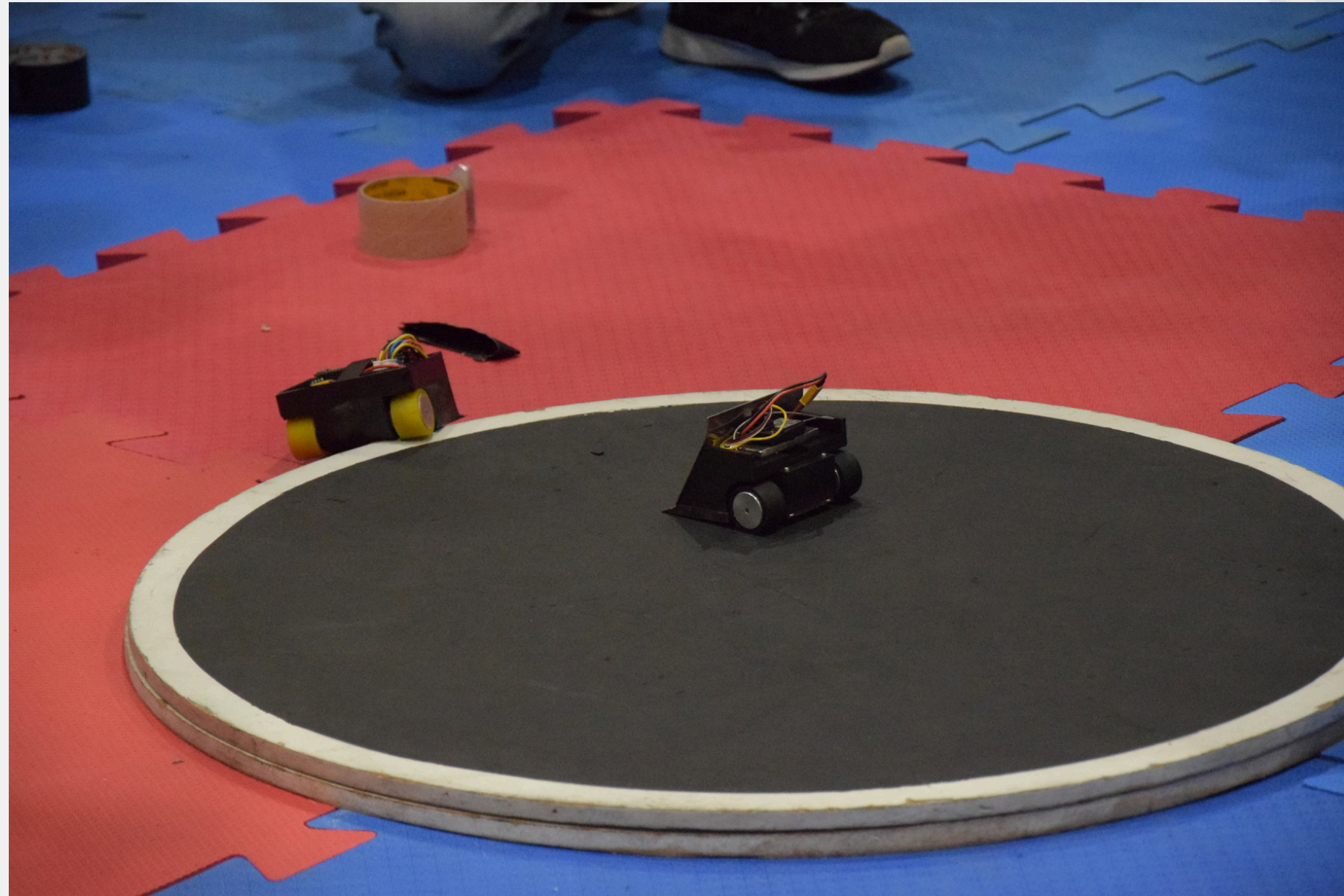


GITHUB:



<https://github.com/PatoBots/Curso-Introducao-Robotica-Movel/tree/Curso-2025-1>

Modalidades:



Sumô



Seguidor de Linha



MOTORES

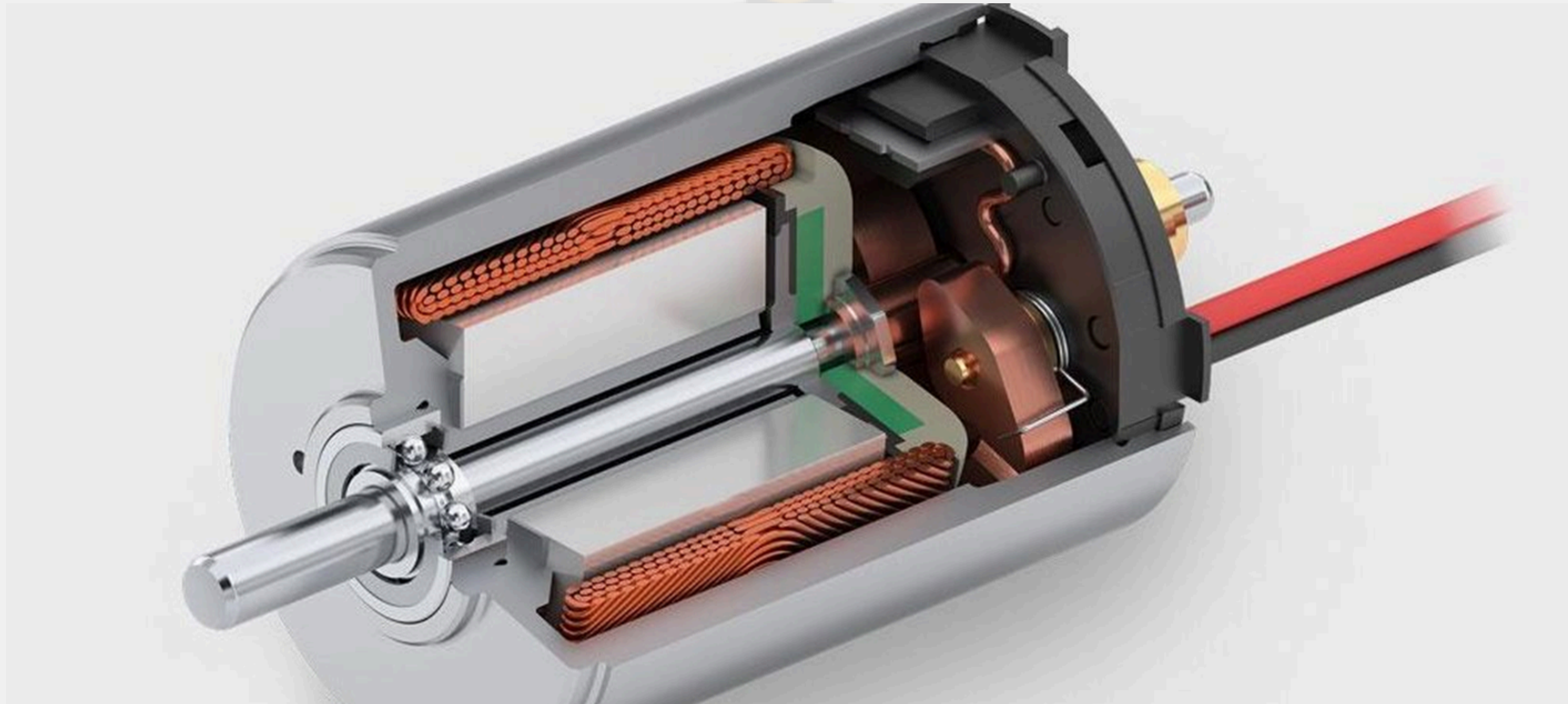


Motor DC (Corrente Contínua)

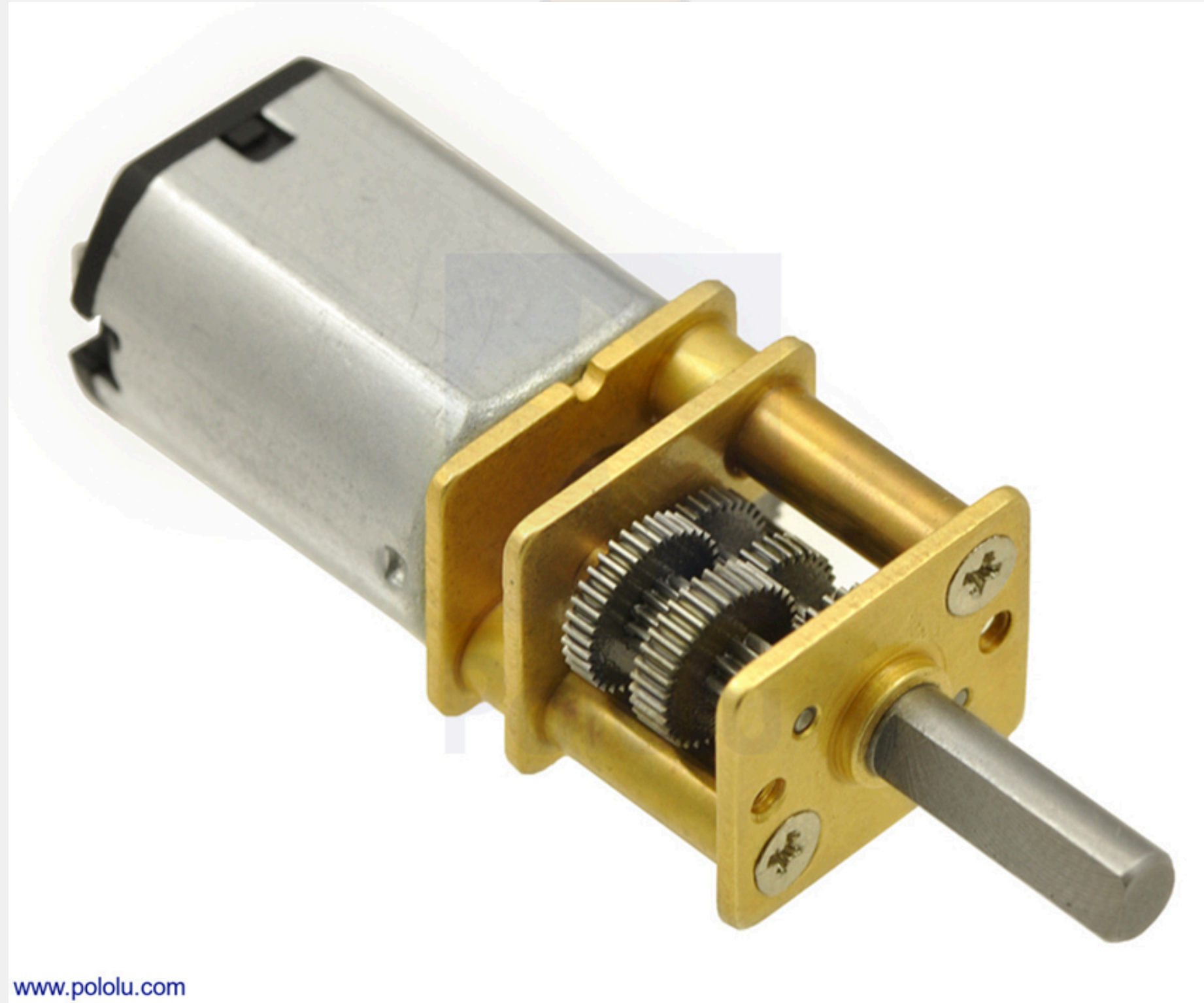


- Conversão de energia:
- Elétrica → Mecânica rotacional.
- Mecanismo:
- Corrente nos enrolamentos do rotor gera campo magnético.
- Interação com campo do estator (ímãs permanentes/eletroímãs).
- Produz torque e rotação.

Motor DC (Corrente Contínua)



Motor DC (Corrente Contínua)





Ponte H L298N

Função:

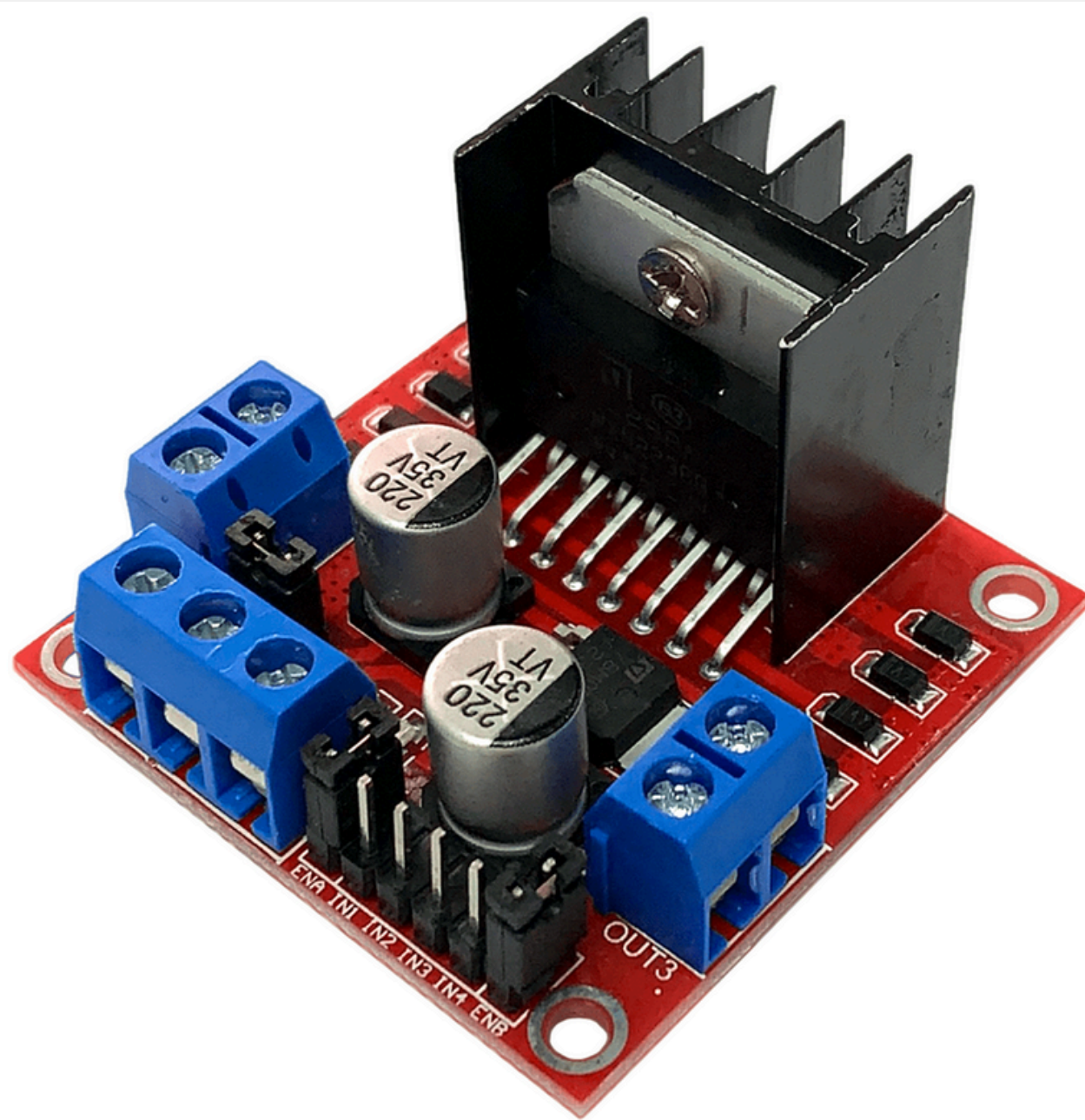
- CI de ponte H dupla para controle de motores.

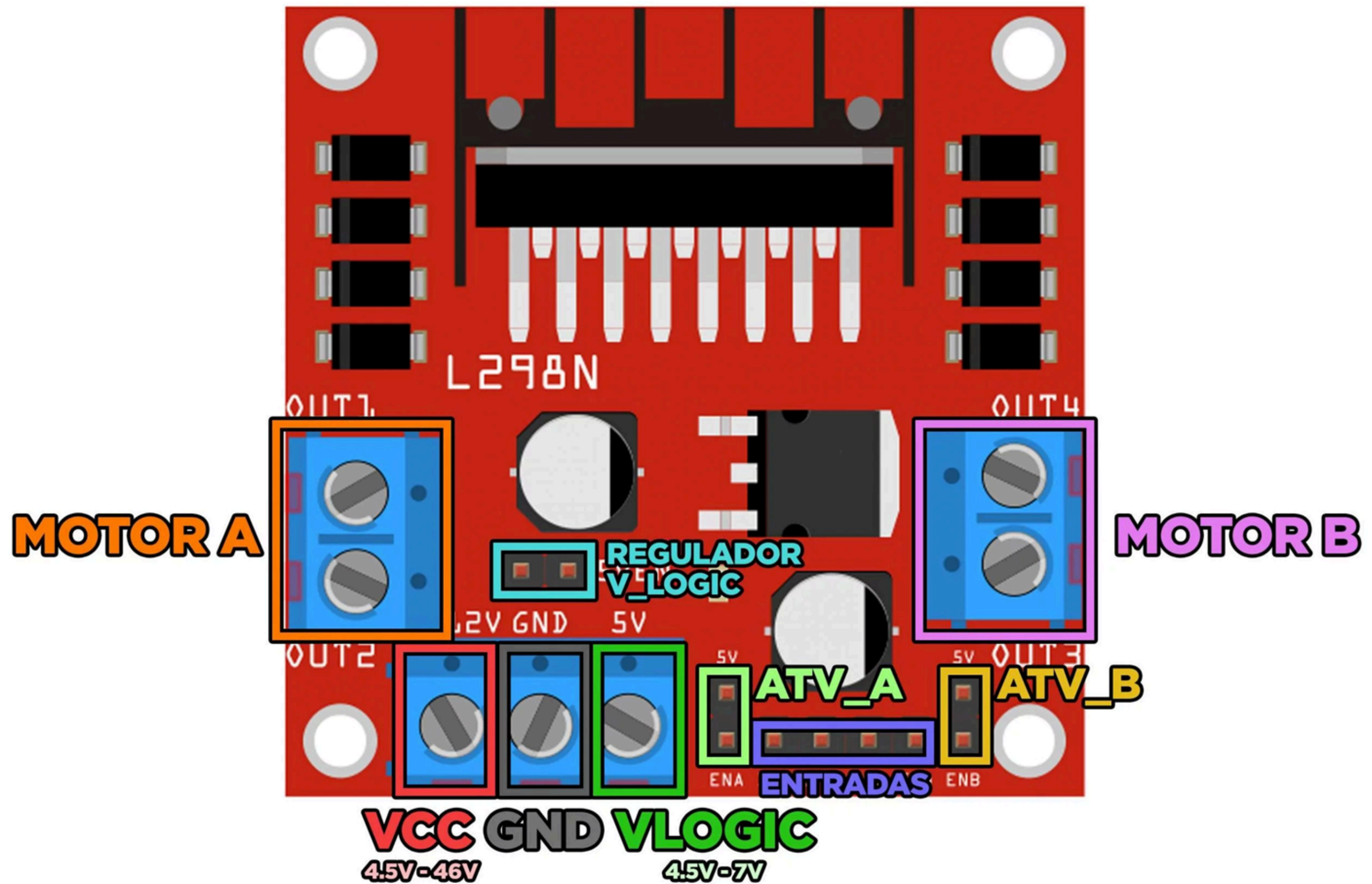
Aplicações:

- Motores DC (até 2)
- Motores de passo (1 bipolar)
- Solenoides

Especificações:

- Tensão: 5–46 V
- Corrente: 2 A contínuos / 3 A pico por canal

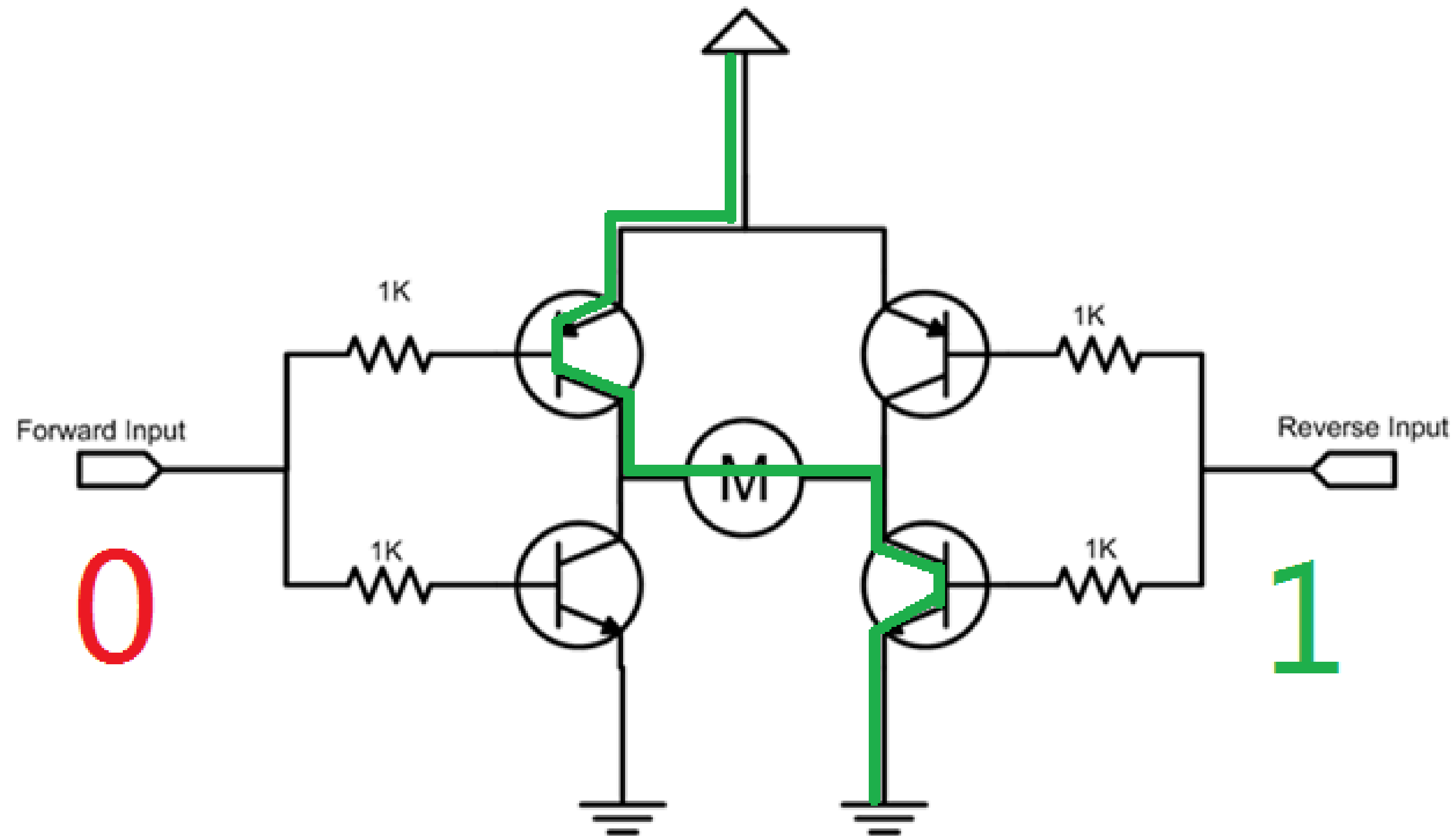




Ponte H L298N

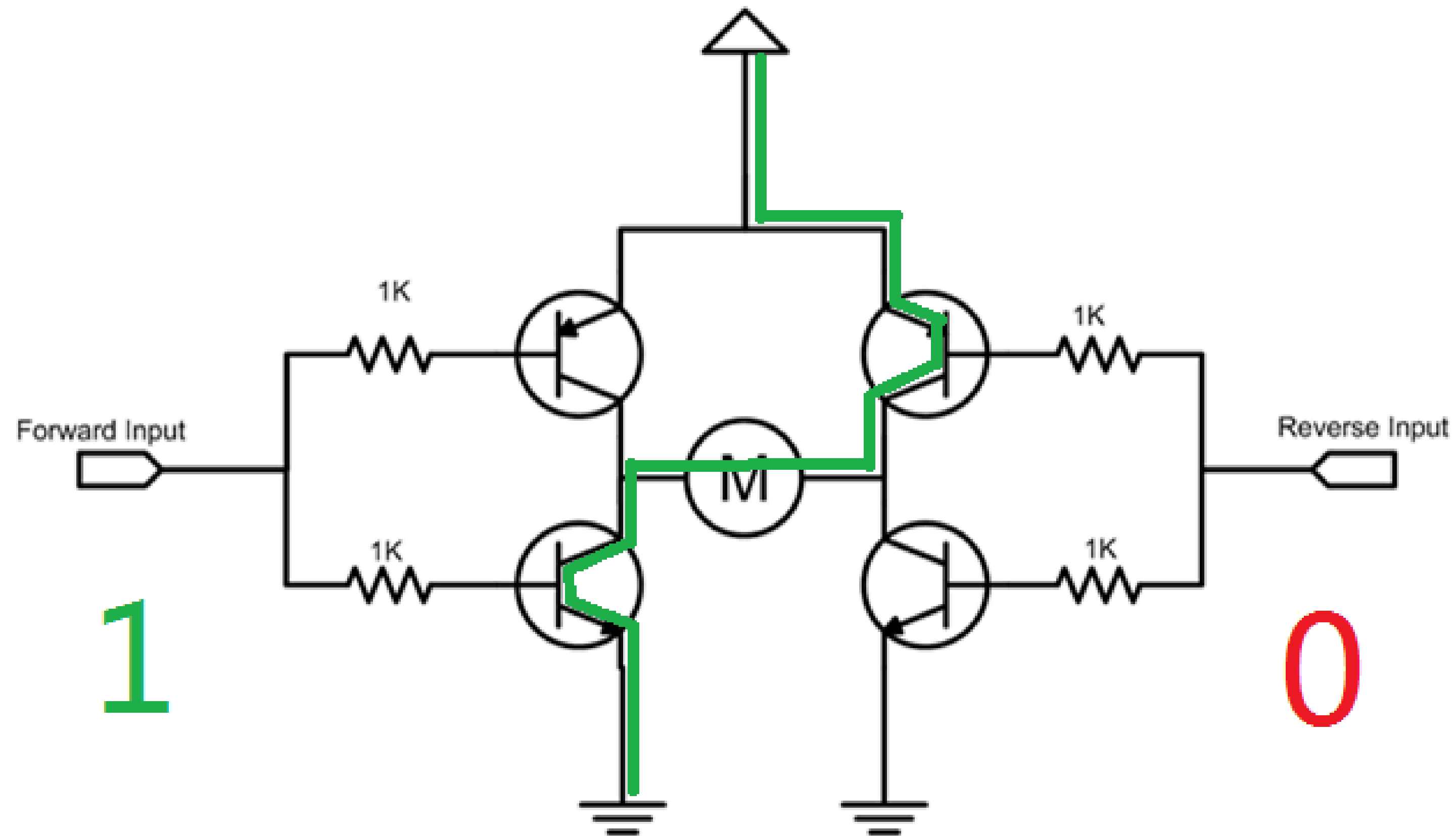
Pino	Função
VS	Alimentação dos motores (6-46V)
VCC	Lógica (5V)
GND	Terra
OUT1/2	Saídas Motor A
OUT3/4	Saídas Motor B
IN1-4	Controle de direção
ENA/ENB	Ativação PWM (velocidade)



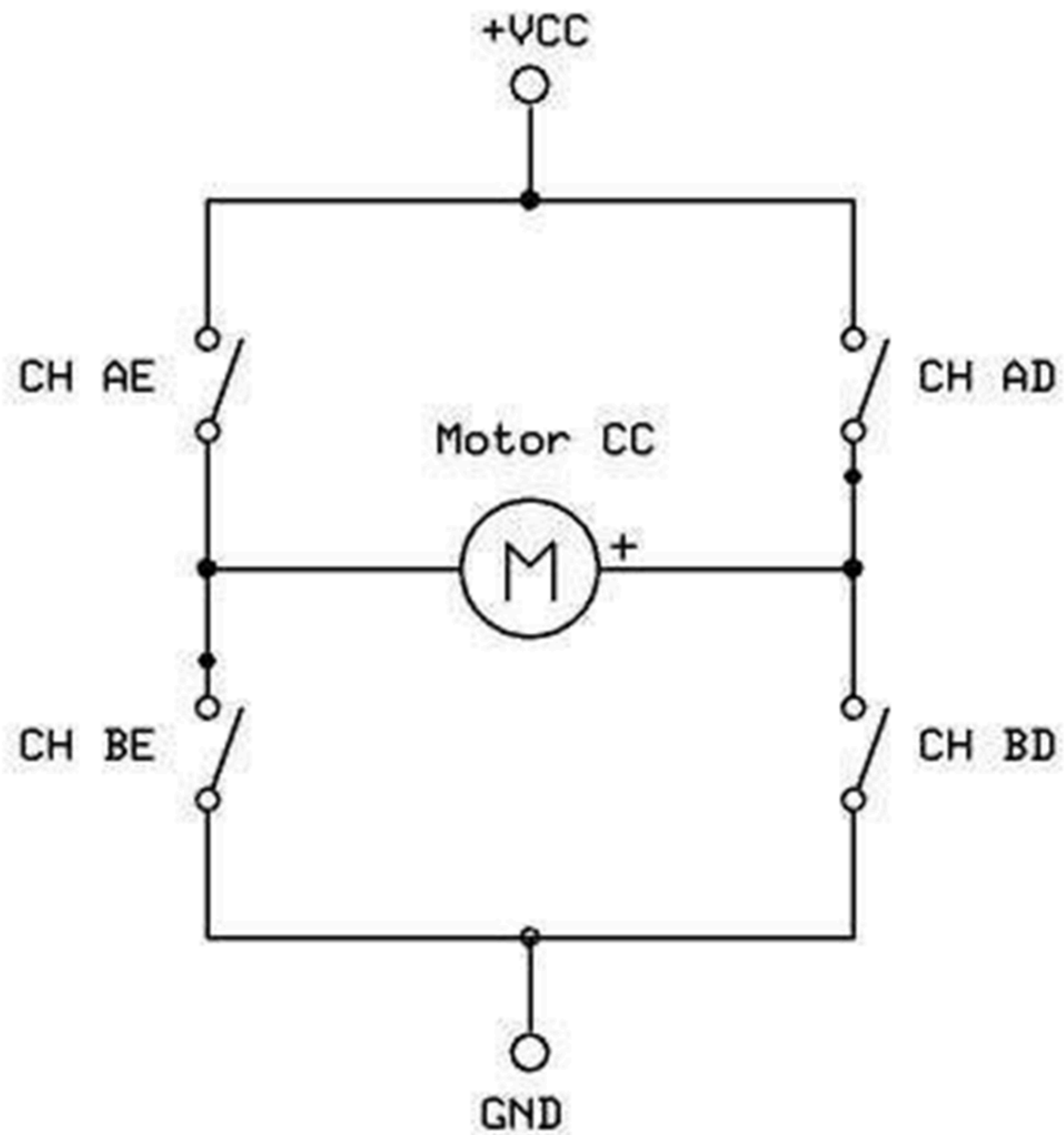


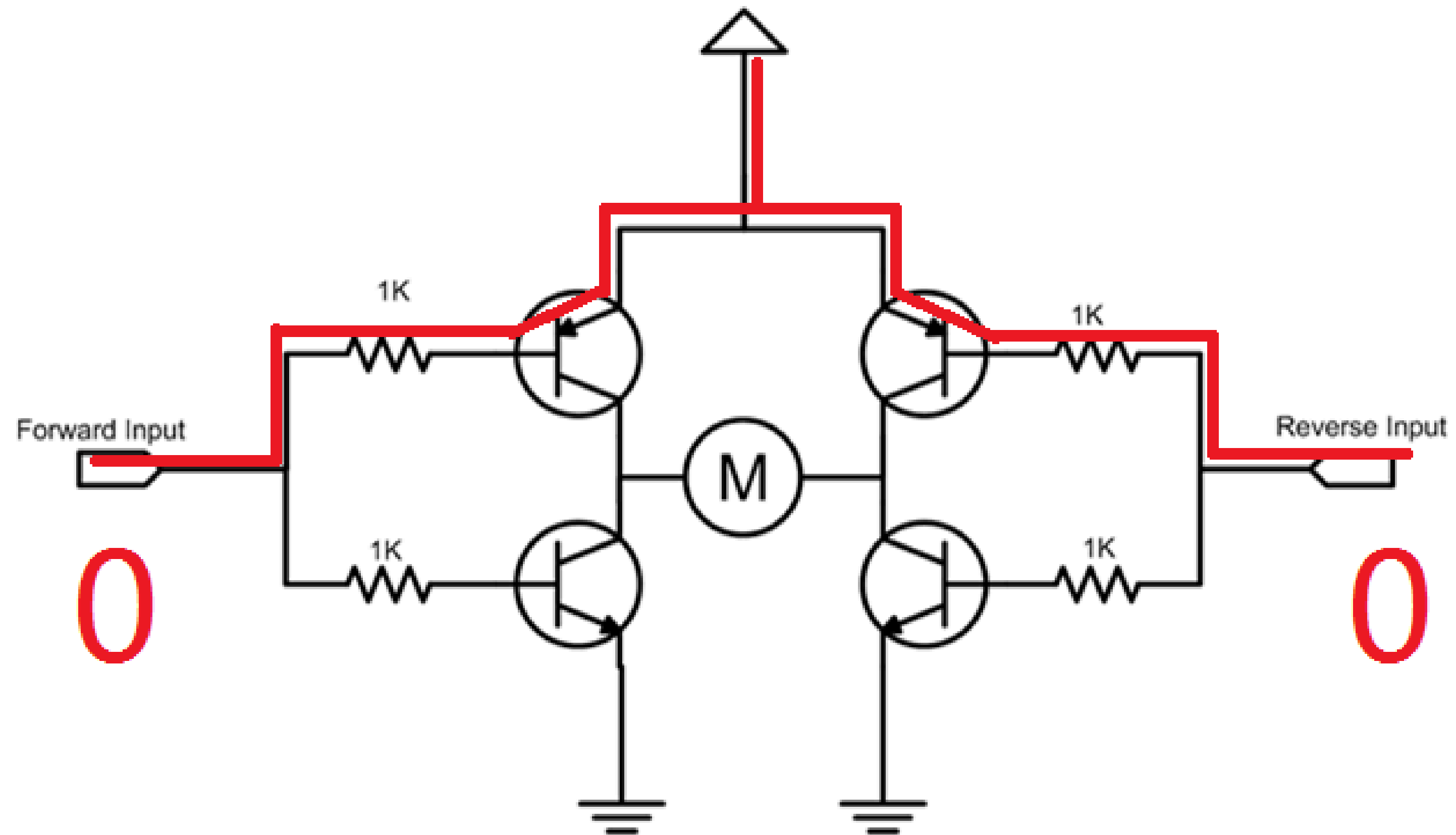
IN1=LOW, IN2=HIGH → Sentido anti-horário

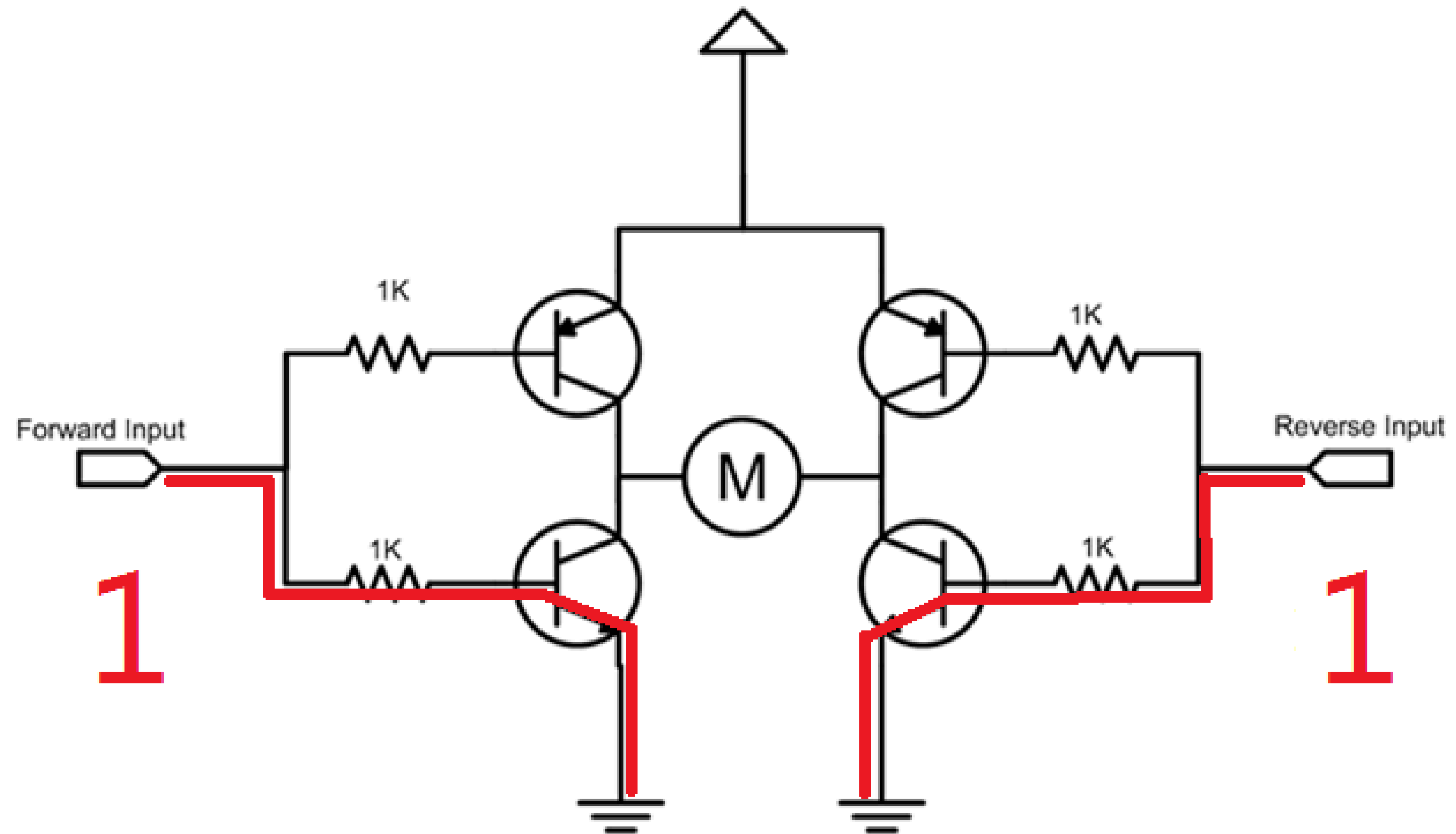
```
1  #define IN1 9
2  #define IN2 10
3
4  void setup() {
5      pinMode(IN1, OUTPUT);
6      pinMode(IN2, OUTPUT);
7  }
8
9  void loop() {
10     digitalWrite(IN1, HIGH);
11     digitalWrite(IN2, LOW);
12 }
```



IN1=HIGH, IN2=LOW → Sentido horário







Ponte H L298N

- Funcionamento:
 - 4 transistores controlam polaridade do motor:
 - IN1=HIGH, IN2=LOW → Sentido horário
 - IN1=LOW, IN2=HIGH → Sentido anti-horário
- Controle de velocidade:
 - PWM nos pinos ENA/ENB (0-255 no Arduino).

Ponte H L298N

Proteções embutidas:

- Diodos flyback (anti-EMF)
- Thermal shutdown (desligamento por superaquecimento)

Problema

Queda de tensão $\sim 2V$

Aquecimento $> 1A$

Ruído elétrico

Solução

Use tensão 20% acima da nominal

Dissipador de calor obrigatório

Filtro RC nos sensores



PWM

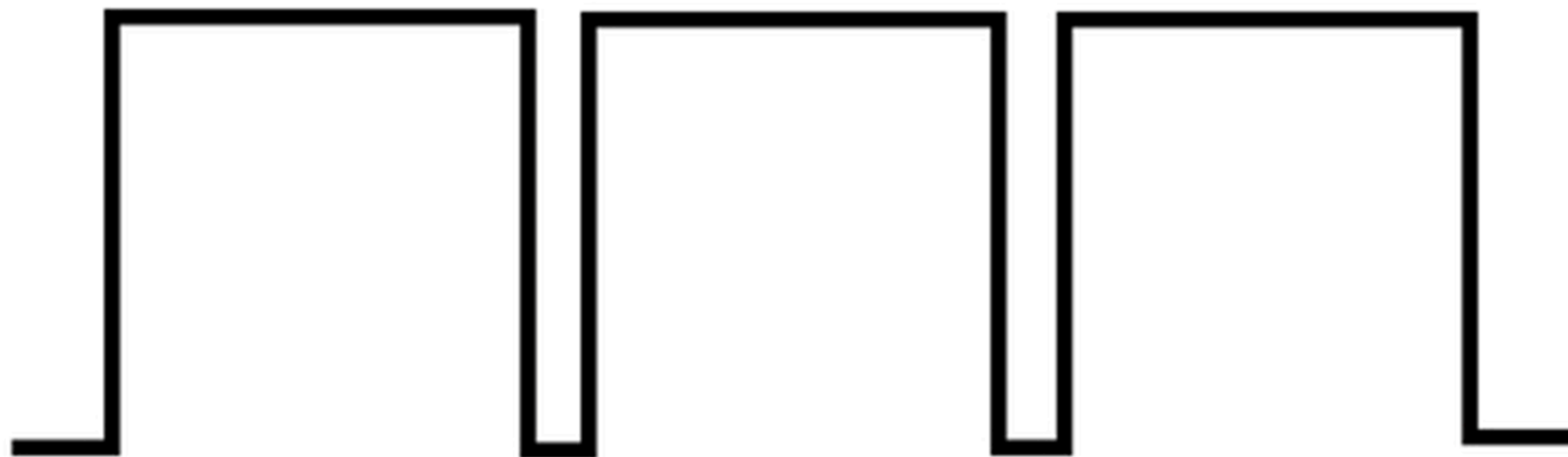
Modulação por Largura de Pulso

Conceito Fundamental

- O que é?: Técnica para simular tensão analógica usando sinal digital.
- Mecanismo:
- Gera um sinal que alterna rapidamente entre HIGH (5V/3.3V) e LOW (0V).
- Duty cycle: Porcentagem do tempo em que o sinal fica em HIGH durante um ciclo.

+VCC

0V



Largura do Pulso

$t_{(on)}$



$t_{(off)}$

Modulação por Largura de Pulso

Cálculo da Tensão Média

- Exemplo (5V):
 - Duty 25% → $5V \times 0.25 = 1.25V$
 - Duty 50% → $5V \times 0.50 = 2.5V$
 - Duty 75% → $5V \times 0.75 = 3.75V$

Modulação por Largura de Pulso

Como Funciona na Prática

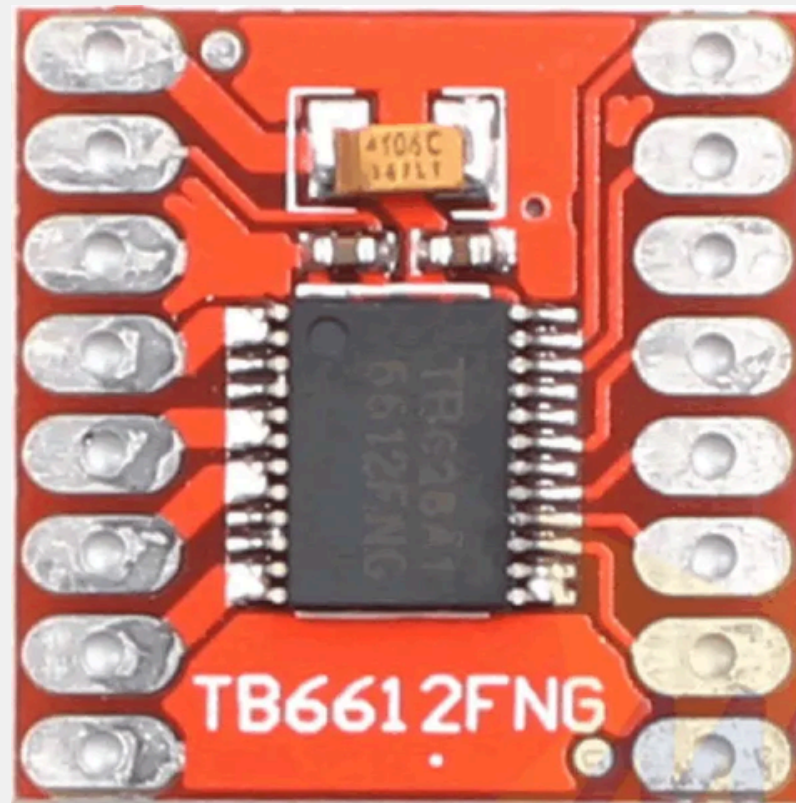
Controle de motor DC:

- Duty cycle 100% → motor a velocidade máxima.
- Duty cycle 30% → motor a ~30% da velocidade.

```
1 void setup() {  
2   pinMode(9, OUTPUT);  
3 }  
4  
5 void loop() {  
6   analogWrite(9, 128);  
7 }
```

```
1 // Pinos do Motor
2 const int IN1 = 7;    // Direção 1
3 const int IN2 = 8;    // Direção 2
4 const int ENA = 9;    // Controle PWM
5
6 void setup() {
7     pinMode(IN1, OUTPUT);
8     pinMode(IN2, OUTPUT);
9     pinMode(ENA, OUTPUT);
10
11     // Configura direção fixa
12     digitalWrite(IN1, HIGH);
13     digitalWrite(IN2, LOW);
14 }
15
16 void loop() {
17     analogWrite(ENA, 64); // 25% - RPM baixo
18
19 }
```


Ponte H TB6612FNG



Função:

- Driver dual para motores DC ou um motor de passo bipolar, baseado em MOSFETs.
- Alta eficiência (91–95%) e baixa queda de tensão (0.3V vs. 1.4V do L298N) .
- Corrente contínua: 1.2A por canal (pico: 3.2A) .
- Tensão de operação: 2.5–13.5V para motores;

Ponte H TB6612FNG

Função STBY (Standby Global)

- Propósito:
 - Reduz consumo para $<1 \mu A$ em modo standby (ideal para projetos com bateria) .
 - Protege contra operação acidental.
- Uso Prático:
 - Sempre ativo no setup() do Arduino:
 - Se omitido, os motores não funcionarão, mesmo com outros comandos.

```
1 void setup() {  
2   pinMode(STBY, OUTPUT);  
3   digitalWrite(STBY, HIGH); // Ativa o driver  
4 }
```

Desafio

Sequência de movimentos:

- * 1. Andar para frente
- * 2. Girar 180° para esquerda
- * 3. Andar para frente novamente
- * 4. Girar 180° para direita
- * 5. Parar completamente

(Tem que voltar para o mesmo local que começou)

```
1 // Definição dos pinos - Motor A (ESQUERDA)
2 #define PWMA 5 // Controle PWM do motor A (velocidade)
3 #define AI1 9 // Controle de direção 1
4 #define AI2 4 // Controle de direção 2
5
6 // Definição dos pinos - Motor B (DIREITA)
7 #define PWMB 6 // Controle PWM do motor B (velocidade)
8 #define BI1 7 // Controle de direção 1
9 #define BI2 8 // Controle de direção 2
10
11 #define STBY 3 // Pino Standby (habilita/desabilita a ponte H)
12
13 void setup() {
14     // Configura todos os pinos como saída
15     pinMode(PWMA, OUTPUT);
16     pinMode(AI1, OUTPUT);
17     pinMode(AI2, OUTPUT);
18
19     pinMode(PWMB, OUTPUT);
20     pinMode(BI1, OUTPUT);
21     pinMode(BI2, OUTPUT);
22
23     pinMode(STBY, OUTPUT);
24
25     // Habilita a ponte H (tira do modo standby)
26     digitalWrite(STBY, HIGH);
27 }
```

```
29 void loop() {
30     // 1. MOVER PARA FRENTE (ambos motores no mesmo sentido)
31     // Motor esquerdo (A)
32     digitalWrite(AI1, HIGH); // Define direção para frente
33     digitalWrite(AI2, LOW);  // O outro pino deve ficar LOW
34     analogWrite(PWMA, 150);  // Velocidade média (~60% da máxima)
35
36     // Motor direito (B)
37     digitalWrite(BI1, HIGH); // Mesma direção do motor A
38     digitalWrite(BI2, LOW);
39     analogWrite(PWMB, 150);
40     delay(2000);              // Move por 2 segundos
41
42     // 2. GIRO 180° ESQUERDA (roda esquerda para trás, direita para frente)
43     // Motor esquerdo para trás
44     digitalWrite(AI1, LOW);   // Inverte a direção
45     digitalWrite(AI2, HIGH);
46     analogWrite(PWMA, 150);
47
48     // Motor direito continua para frente
49     digitalWrite(BI1, HIGH);
50     digitalWrite(BI2, LOW);
51     analogWrite(PWMB, 150);
52     delay(1000);              // Tempo de giro (ajustar conforme necessário)
53 }
```

```

54 // 3. MOVER PARA FRENTE NOVAMENTE
55 // Ambos motores para frente
56 digitalWrite(AI1, HIGH);
57 digitalWrite(AI2, LOW);
58 analogWrite(PWMA, 150);
59 digitalWrite(BI1, HIGH);
60 digitalWrite(BI2, LOW);
61 analogWrite(PWMB, 150);
62 delay(2000);
63
64 // 4. GIRO 180° DIREITA (roda direita para trás, esquerda para frente)
65 // Motor esquerdo para frente
66 digitalWrite(AI1, HIGH);
67 digitalWrite(AI2, LOW);
68 analogWrite(PWMA, 150);
69
70 // Motor direito para trás
71 digitalWrite(BI1, LOW); // Inverte a direção
72 digitalWrite(BI2, HIGH);
73 analogWrite(PWMB, 150);
74 delay(1000);
75
76 // 5. PARADA COMPLETA
77 analogWrite(PWMA, 0); // Desliga PWM (velocidade zero)
78 analogWrite(PWMB, 0);
79
80 while (1); // Loop infinito para parar a execução
81 // Remova esta linha se quiser que a sequência se repita
82 }

```


Comparação

Característica	TB6612FNG	L298N
Tecnologia	MOSFETs (baixa resistência)	Transistores bipolares (BJT)
Eficiência energética	90–95% (quase zero perda de tensão)	40–70% (queda de 1.4–2V)
Queda de tensão	0.3V (como resistor de 0.5Ω)	1.4–2V (perda significativa)
Corrente contínua	1.2A por canal (pico: 3.2A)	2A por canal (pico: 3A)
Tensão de operação	2.5–13.5V (motores) 2.7–5.5V (lógica)	4.5–46V (amplo alcance)
Dissipação de calor	Não requer dissipador	Exige dissipador grande
Modo standby	Sim (baixo consumo)	Não
Tamanho físico	Compacto (≈20mm²)	Grande (com dissipador)
Proteções integradas	Térmica e contra corrente reversa	Diodos flyback externos

A large, faint watermark of the character Pato Bot is centered in the background. It shows the robot's head with two large circular eyes, its torso with a yellow and orange suit, and its legs. At the bottom of the watermark is the text 'pato bots' in a stylized, blocky font.

REGULADOR DE TENSÃO STEP-UP

Regulador de Tensão Step-Up



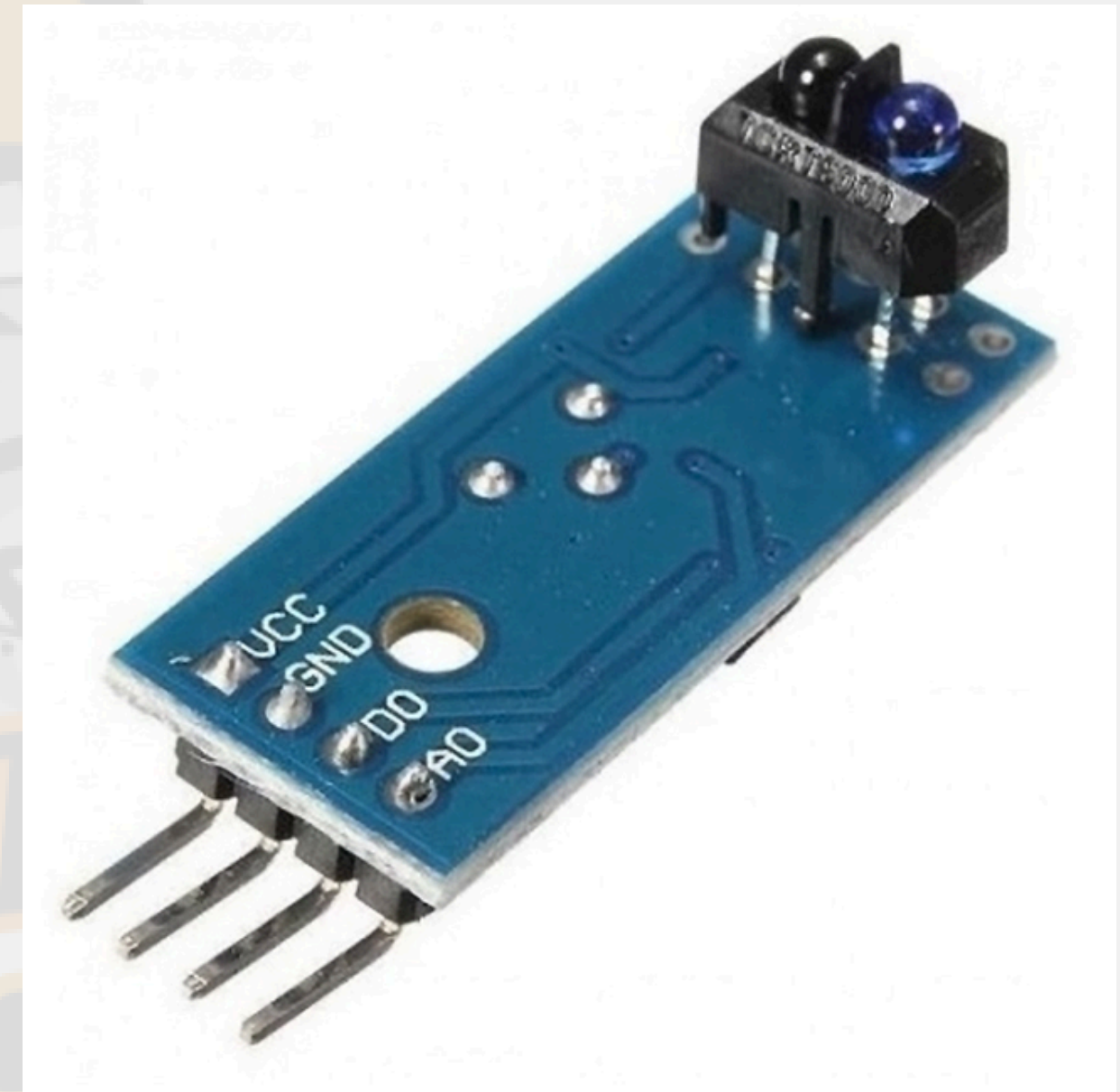
Regulador de Tensão Step-Up

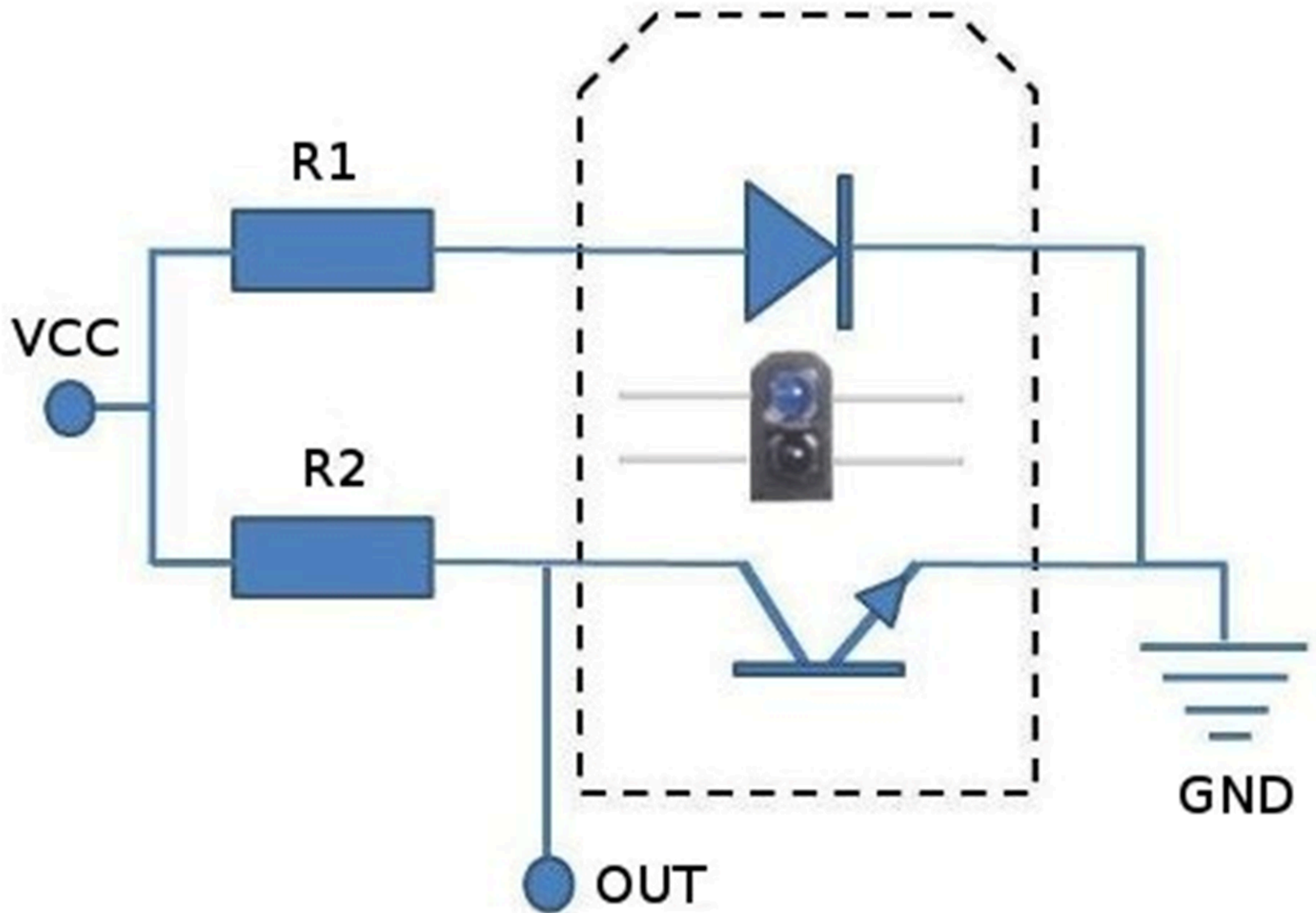
- Dispositivo que tem como base semicondutores, tais como diodos e circuitos integrados, que tem a função de aumentar ou diminuir a tensão de saída de um circuito elétrico.
- Um regulador de tensão é incapaz de gerar energia.

SENSOR ÓPTICO REFLEXIVO



Sensor Óptico Reflexivo





Sensor Óptico Reflexivo

- O sensor tem dois componentes: Um sensor infravermelho (Azul) e um fototransistor (Preto) divididos por uma divisória preta.
- Quando algum objeto se aproxima do sensor, a luz infravermelha é refletida no objeto, passa para o outro lado, ativando o resistor e fazendo a leitura.
- Quanto maior reflexivo o material (como por exemplo a cor branca), maior será o sinal enviado, e quanto menos reflexivo (como por exemplo a cor preta), o sinal enviado será próximo de zero.

Sensor Óptico Reflexivo

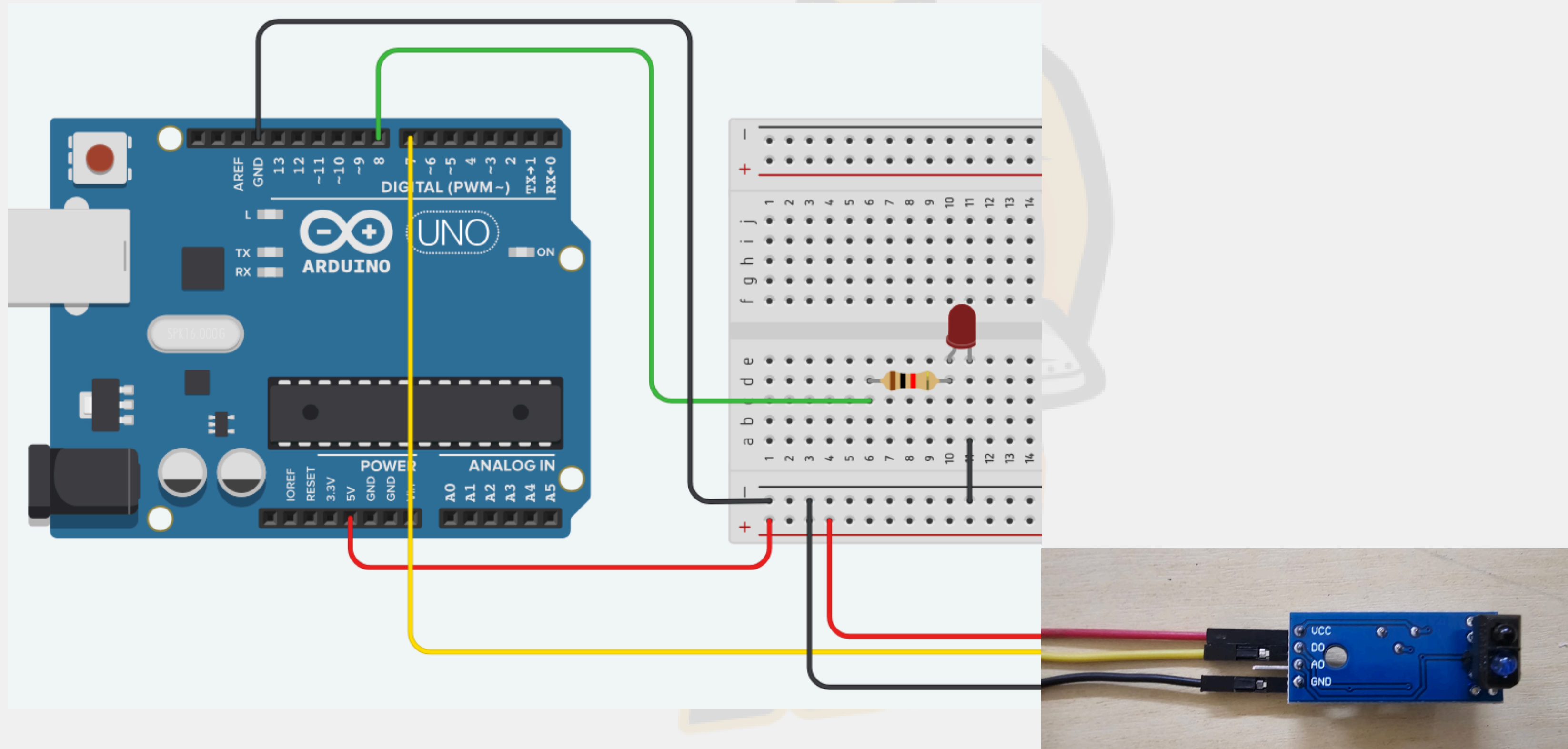
- Ele tem dois modos de leitura: **Analógico** e **Digital**.
- O **Modo Analógico** vai enviar a quantidade de reflexão que o sensor está lendo.
- O **Modo Digital** vai enviar se o sensor está lendo a reflexão ou não.

Sensor Óptico Reflexivo

- Este sensor óptico reflexivo está soldado em uma placa que faz aumentar a distância qual o sensor consegue refletir seu infravermelho, também acertando uma maior precisão.
- Na parte de cima da placa tem um potenciômetro que ajusta a precisão da leitura do sensor, regulando o quanto de sinal ele envia pela distância.

Sensor Óptico Reflexivo

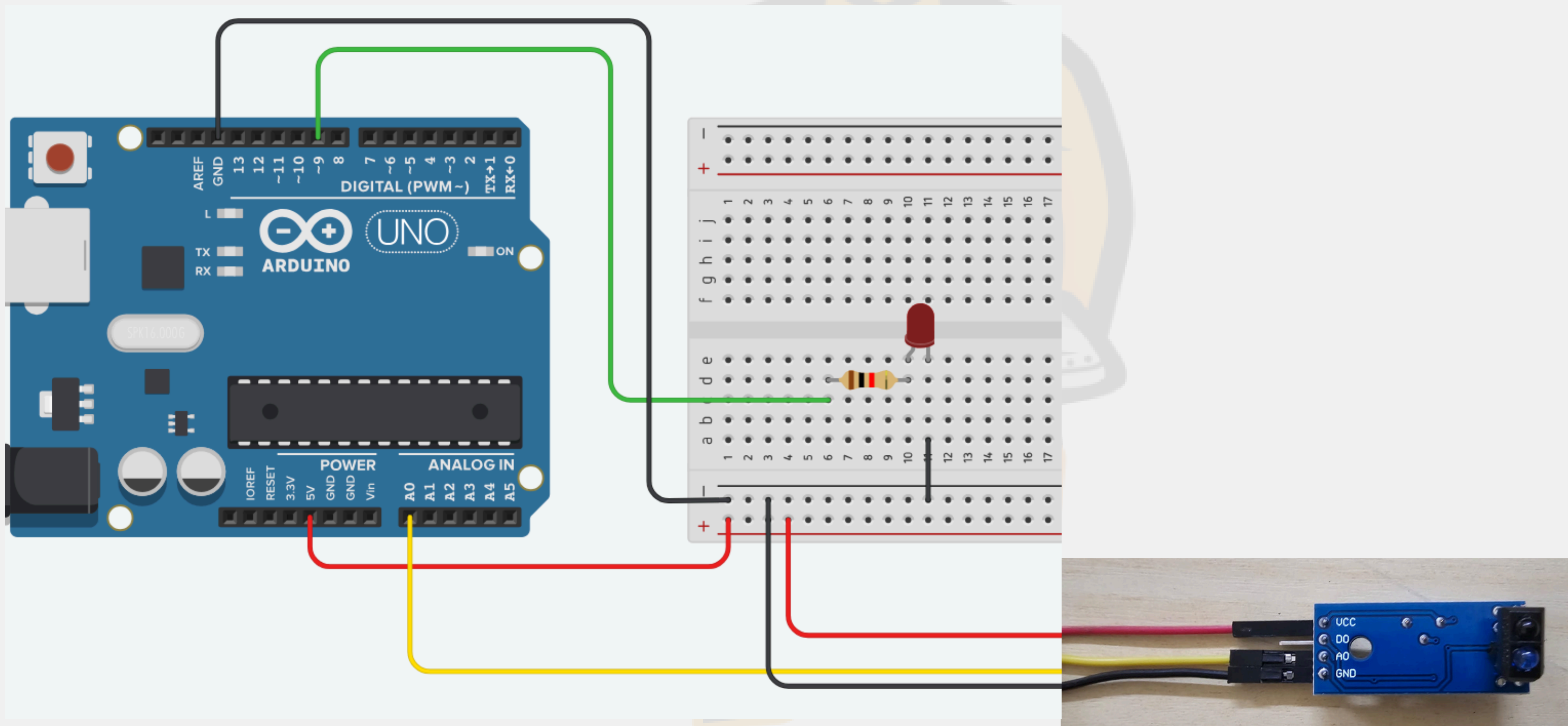
Exemplo Digital



```
1 int led = 8; //Pino do led
2 int sensor = 7; //Ligado ao pino "coletor" do sensor óptico
3 int leitura = 0; //Armazena informações sobre a leitura do sensor
4
5 void setup()
6 {
7     pinMode(led, OUTPUT); //Define o pino do led como saída
8     pinMode(sensor, INPUT); //Define o pino do sensor como entrada
9 }
10
11 void loop()
12 {
13     //Le as informações do pino do sensor
14     leitura = digitalRead(sensor);
15
16     if (leitura != 1) //Verifica se o objeto foi detectado
17     {
18         digitalWrite(led, 1);
19     }
20     else{
21         digitalWrite(led, 0);
22     }
23 }
```

Sensor Óptico Reflexivo

Exemplo Analógico




```
1  int led = 9; // Pino do led;
2  int sensor = 0; // Ligado ao pino "coletor" do sensor óptico
3  int leitura, valor; // Armazena informações sobre a leitura do sensor
4
5  void setup() {
6      Serial.begin(9600);
7      pinMode(led, OUTPUT); // Define o pino do led como saída
8      pinMode(sensor, INPUT); // Define o pino do sensor como entrada
9  }
11 void loop() {
12     //Le as informações do pino do sensor
13     leitura = analogRead(sensor);
14     Serial.println(leitura);
15
16     //Verifica se o objeto foi detectado
17     if(leitura>800)
18     |   valor = 0;
19     else if(leitura<800 && leitura >= 600)
20     |   valor = 50;
21     else if(leitura<600 && leitura >= 400)
22     |   valor = 100;
23     else if(leitura < 400 && leitura >= 200)
24     |   valor = 150;
25     else if(leitura < 200)
26     |   valor = 250;
27
28     analogWrite(led, valor);
29 }
```

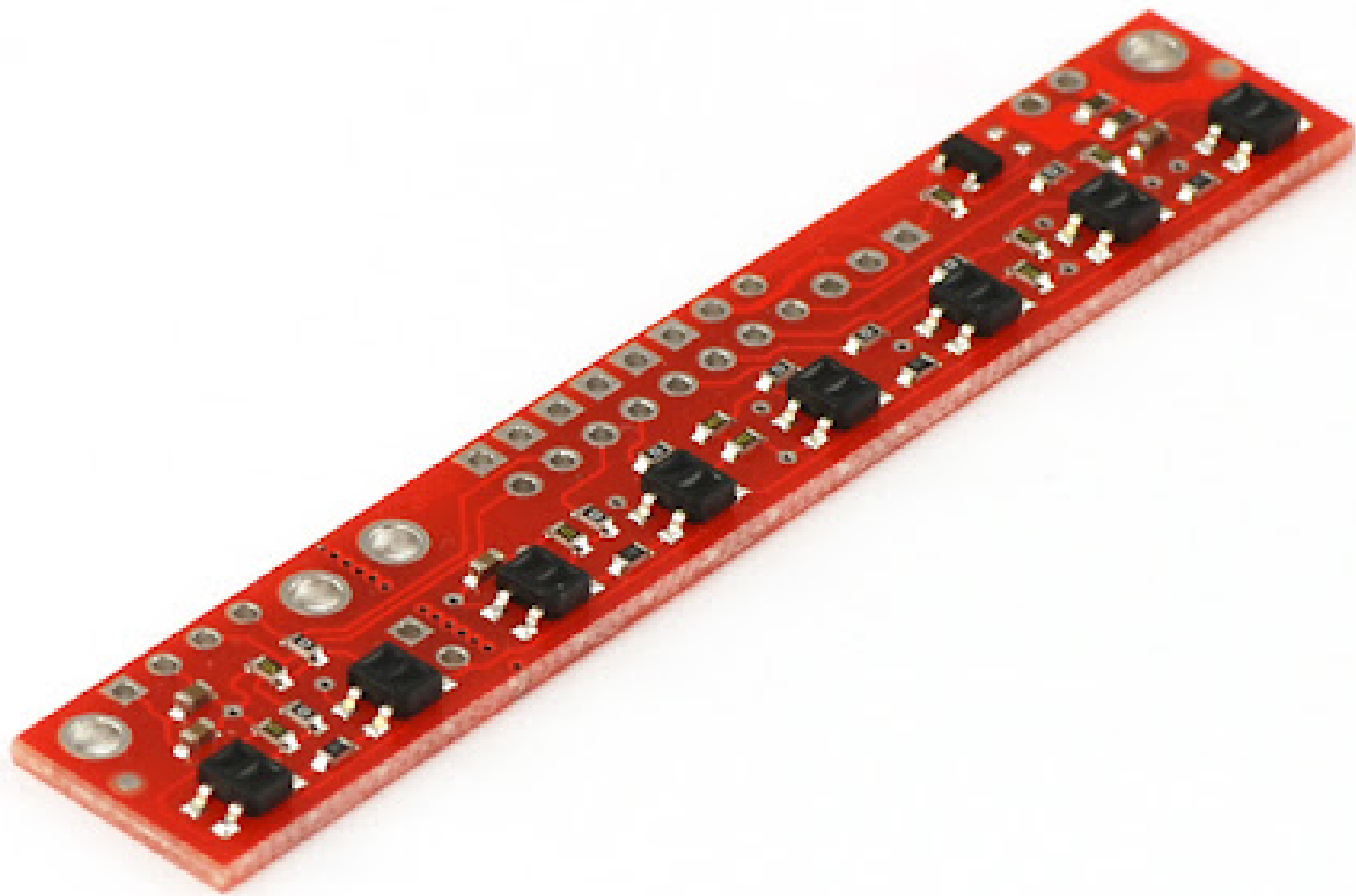
SENSOR DE REFLETÂNCIA QTR



Sensor de Refletância QTR

QTR-8 A/D

- Contém oito pares de emissores e receptores infravermelhos (LEDs e fototransistores).
- Retorna um sinal do sensor que detectou a reflexão.

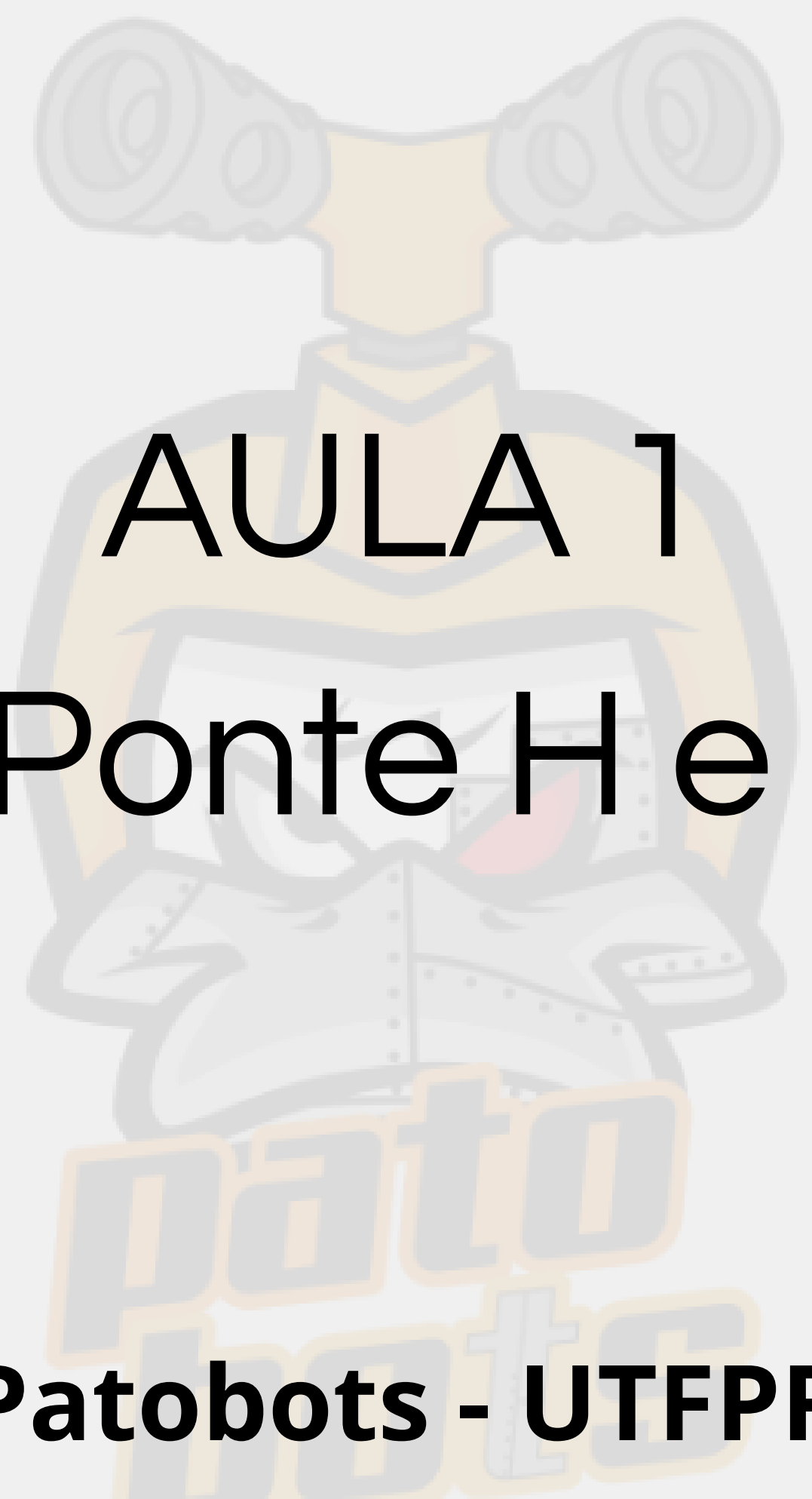


Sensor de Refletância QTR



QTR-1 A/D

- Serve como um Sensor Óptico Reflexivo, porém tem melhor resposta e velocidade.
- O Sensor de Refletância também pode devolver um sinal **Analógico** ou **Digital** com a mesma função.



AULA 1

Motores, Ponte H e Sensores

Patobots - UTFPR